Classification of Digital Data

1. Structured Data

- **Definition:**
    - Data that is highly organized and stored in a predefined format (rows and columns).
    - Easily searchable within databases using SQL queries.
    - Typically managed using relational database management systems (RDBMS).
- **Examples:**
    - Customer records in an e-commerce database (Name, Email, Address, Purchase History).
    - Financial transactions in banking systems.
    - Employee data in an HR management system.
- **Advantages:**
    - High accuracy due to structured format.
    - Easy to search, retrieve, and analyze using SQL queries.
    - Well-suited for transactional processing and business intelligence.
- **Disadvantages:**
    - Limited flexibility as it requires predefined schema.
    - Cannot handle complex, multimedia, or real-time data efficiently.
    - Scaling structured databases can be expensive and challenging.
- **Usage:**
    - Banking systems for managing customer accounts and transactions.
    - Enterprise Resource Planning (ERP) systems for business processes.
    - Inventory management in retail and supply chains.

2. Unstructured Data

- **Definition:**
    - Data that does not follow a predefined schema or format.
    - Can include text, images, videos, audio files, social media posts, etc.
    - Requires advanced processing techniques like Natural Language Processing (NLP) or Image Recognition for analysis.
- **Examples:**
    - Emails, chat logs, and social media content.
    - Video and image files on YouTube or Instagram.
    - Sensor data from IoT devices.
- **Advantages:**
    - Capable of storing and processing rich and diverse data types.
    - More comprehensive insights as it includes text, images, and videos.
    - Suitable for real-time data sources like live video feeds.
- **Disadvantages:**
    - Hard to analyze due to the lack of predefined structure.
    - Requires advanced tools like AI, ML, and deep learning for processing.
    - Storage and retrieval are complex and may require NoSQL databases or cloud solutions.
- **Usage:**
    - Social media analysis for sentiment detection and brand monitoring.
    - Image and video recognition for security and surveillance.
    - Customer feedback analysis in e-commerce and service industries.

3. Semi-Structured Data

- **Definition:**
    - Data that does not conform to a strict tabular structure but still contains some organizational elements like tags and metadata.
    - Often stored in flexible formats like XML, JSON, or YAML.
    - Bridges the gap between structured and unstructured data.
- **Examples:**
    - XML and JSON files used in web applications.
    - Log files generated by applications and servers.
    - Emails (which have structured metadata like sender, recipient, timestamp but unstructured message content).
- **Advantages:**
    - More flexible than structured data while still retaining some level of organization.
    - Easier to analyze than purely unstructured data.
    - Can be processed using both SQL-based and NoSQL-based systems.
- **Disadvantages:**
    - Requires specialized parsing techniques to extract meaningful insights.
    - More complex storage and retrieval mechanisms compared to structured data.
    - Processing speed may be slower compared to structured databases.
- **Usage:**
    - Web APIs that exchange data in JSON or XML formats.
    - Email processing systems that extract structured metadata from messages.
    - Log management in cybersecurity for tracking events and incidents.

Characteristics of Big Data (The 7 V's and Beyond)

1. Volume (Size of Data)

- **Definition:**
    - Refers to the enormous amount of data generated every second from multiple sources.
    - Traditional databases struggle to store and process such massive datasets.
- **Example:**
    - Facebook generates **over 4 petabytes** of data daily, including posts, images, and videos.

2. Velocity (Speed of Data Generation and Processing)

- **Definition:**
    - Describes how quickly data is created, collected, and processed.
    - Many applications require real-time or near-real-time data processing.
- **Example:**
    - Stock market trading systems process **millions of transactions per second**, requiring ultra-fast analytics.

3. Variety (Diverse Data Formats)

- **Definition:**
    - Data comes in different forms – structured, semi-structured, and unstructured.
    - Traditional systems can only handle structured data efficiently.
- **Example:**
    - Emails (semi-structured), videos (unstructured), and SQL databases (structured) co-exist in organizations.

4. Veracity (Data Quality and Accuracy)

- **Definition:**
    - Refers to inconsistencies, biases, and errors in data.
    - Poor-quality data can lead to misleading insights.
- **Example:**
    - A company analyzing social media sentiment may get **biased results due to fake or bot-generated reviews**.

5. Value (Extracting Meaningful Insights)

- **Definition:**
    - The usefulness and relevance of data determine its business value.
    - Collecting data without deriving actionable insights is wasteful.
- **Example:**
    - Retailers use big data to **predict customer preferences** and enhance personalized marketing strategies.

6. Variability (Inconsistencies in Data Flow)

- **Definition:**
    - Data patterns and formats keep changing, making it difficult to maintain consistency.
    - Seasonal trends, context shifts, and unexpected anomalies affect data analysis.
- **Example:**
    - **Customer demand for e-commerce platforms fluctuates** heavily during festivals and sales events.

7. Visualization (Making Data Understandable)

- **Definition:**
    - Refers to presenting large volumes of complex data in an interpretable format using charts, dashboards, and graphs.
- **Example:**
    - **COVID-19 tracking dashboards** visualize cases, recoveries, and vaccination trends for better decision-making.

Additional V's in Big Data:

Apart from the core **7 V's**, researchers and experts have introduced more characteristics to describe Big Data comprehensively:

- **Volatility:** Data relevance changes over time (e.g., trending hashtags on Twitter).
- **Validity:** Ensuring data is collected from credible sources (e.g., verified health records vs. unreliable internet articles).
- **Vulnerability:** Data security concerns, including cyber threats and breaches (e.g., financial fraud detection).
- **Venue:** The geographical origin of data (e.g., weather data from different countries may vary in format and accuracy).

The Need and Evolution of Big Data

Why Do We Need Big Data?

1. **Explosion of Data Growth**
    - Traditional methods cannot handle the **massive increase in digital data** from social media, IoT, and smart devices.
2. **Competitive Advantage for Businesses**
    - Companies that leverage Big Data **gain insights into customer behavior**, leading to better decision-making.

3. **Scientific and Medical Advancements**
    - Genomic research, drug discovery, and personalized medicine rely on **analyzing vast amounts of biological data**.
4. **Fraud Detection and Cybersecurity**
    - Real-time analytics help detect fraudulent transactions and identify cybersecurity threats.
5. **Smart Cities and IoT**
    - Traffic management, energy consumption, and disaster prediction depend on **continuous sensor data analysis**.

Evolution of Big Data

- **Early Data Storage & Processing (1950s - 1990s)**
    - Mainframes and relational databases handled structured data but struggled with large volumes.
- **Rise of the Internet (2000s)**
    - Websites, e-commerce, and social media led to an exponential increase in unstructured data.
- **Hadoop and NoSQL Revolution (2006 - 2010s)**
    - Google's **MapReduce algorithm inspired Hadoop**, enabling distributed storage and parallel processing.
    - NoSQL databases like MongoDB and Cassandra emerged to handle **unstructured and semi-structured data**.
- **AI & Cloud-based Big Data (2020s - Present)**
    - **Machine learning, deep learning, and cloud computing** enhance Big Data analytics for real-time decision-making.
    - Cloud platforms like AWS, Google Cloud, and Azure provide scalable storage and processing solutions.

Challenges Faced by Big Data

1. Data Storage and Scalability

- **Issue:** Traditional databases struggle to store petabytes of data.
- **Solution:** Distributed file systems (HDFS, Amazon S3) and cloud storage improve scalability.

2. Data Processing and Speed

- **Issue:** Traditional methods cannot process data in real time.
- **Solution:** Apache Spark, Flink, and in-memory computing enhance real-time analytics.

3. Data Quality and Cleaning

- **Issue:** Inconsistent, incomplete, and noisy data affects insights.
- **Solution:** Data preprocessing techniques, data validation, and AI-powered cleaning tools help.

4. Security and Privacy Concerns

- **Issue:** Increasing cyberattacks, identity theft, and data breaches.
- **Solution:** Encryption, access control mechanisms, and GDPR/CCPA compliance improve security.

5. Data Integration from Multiple Sources

- **Issue:** Combining structured, semi-structured, and unstructured data from various sources is challenging.
- **Solution:** ETL (Extract, Transform, Load) pipelines and data lakes manage diverse datasets efficiently.

6. High Infrastructure and Maintenance Costs

- **Issue:** Storing, managing, and processing Big Data requires expensive hardware and skilled personnel.
- **Solution:** Cloud computing and serverless architectures reduce cost overheads.

7. Ethical and Bias Issues in Big Data Analytics

- **Issue:** Algorithmic bias, discrimination, and ethical concerns in AI decision-making.

- **Solution:** Transparent AI models, fairness testing, and ethical guidelines for Big Data usage.

Traditional Business Intelligence (BI) vs. Big Data

1. Data Volume

- **Traditional BI:**
    - Handles **structured** data with limited volume (typically gigabytes to terabytes).
    - Works efficiently with **small to medium-sized datasets** stored in relational databases.
- **Big Data:**
    - Manages **massive datasets** (terabytes to petabytes and beyond).
    - Uses **distributed storage systems** like Hadoop and cloud storage for scalability.

2. Data Variety

- **Traditional BI:**
    - Deals primarily with **structured** data from transactional databases, spreadsheets, and ERP systems.
- **Big Data:**
    - Can handle **structured, semi-structured, and unstructured** data.
    - Works with **text, images, videos, social media posts, IoT sensor data, and logs**.

3. Data Processing Approach

- **Traditional BI:**
    - Uses **batch processing** with pre-defined queries and structured data models.
    - SQL-based querying and **pre-aggregated data reports**.

- **Big Data:**
    - Uses both **batch processing (Hadoop) and real-time streaming (Apache Kafka, Spark Streaming)**.
    - Supports **machine learning, natural language processing (NLP), and AI-driven analytics**.

4. Data Storage & Architecture

- **Traditional BI:**
    - Relies on **centralized databases** (e.g., relational databases like Oracle, MySQL, SQL Server).
    - Uses **Data Warehouses** (e.g., SAP BW, Teradata) for historical analysis.
- **Big Data:**
    - Uses **distributed storage** across multiple nodes (HDFS, NoSQL, Cloud Data Lakes).
    - Data is stored in formats like **JSON, Parquet, Avro, and columnar storage** for flexibility.

5. Data Speed (Velocity & Processing Time)

- **Traditional BI:**
    - Works well for **historical data analysis** but struggles with real-time insights.
    - Queries can take **minutes to hours** to execute, especially on large datasets.
- **Big Data:**
    - Supports **real-time analytics and near-instant insights**.
    - Uses **stream processing frameworks** like Apache Kafka, Flink, and Spark.

6. Scalability

- **Traditional BI:**
    - **Limited scalability**, as traditional databases slow down with increasing data volume.
    - Scaling requires **expensive hardware upgrades** (vertical scaling).
- **Big Data:**
    - **Highly scalable**, as distributed computing allows horizontal scaling (adding more servers).
    - Uses **cloud-based auto-scaling solutions** like AWS, Google Cloud, and Azure.

7. Cost & Infrastructure

- **Traditional BI:**
    - Requires **expensive on-premises servers** and high licensing fees for enterprise software.
    - Costly **ETL (Extract, Transform, Load) processes** for data integration.
- **Big Data:**
    - Uses **open-source tools** (Hadoop, Spark, NoSQL databases) to reduce costs.
    - **Cloud-based storage and computing** allow pay-as-you-go pricing.

8. Analytics Capabilities

- **Traditional BI:**
    - Provides **descriptive and diagnostic analytics** (what happened & why).
    - Uses **dashboards, reports, and OLAP cubes** for business decision-making.
- **Big Data:**
    - Supports **predictive and prescriptive analytics** using AI/ML models.
    - Uses **deep learning, NLP, and data mining** to generate intelligent insights.

9. Decision-Making Approach

- **Traditional BI:**
    - **Static and periodic reporting** for executives and business analysts.
    - Decisions are **based on historical data** rather than real-time information.
- **Big Data:**
    - Enables **dynamic, real-time decision-making** with AI-driven recommendations.
    - Helps businesses adapt to **rapidly changing market trends**.

10. Security & Compliance

- **Traditional BI:**
    - Well-established **security frameworks** with **role-based access control (RBAC)**.
    - **Strict compliance** with regulations like **SOX, HIPAA, GDPR**.
- **Big Data:**
    - Faces **security challenges** due to diverse and distributed storage.
    - Uses **encryption, data masking, and identity management** for compliance.

11. Use Cases & Industries

- **Traditional BI:**
    - **Financial reporting, sales performance analysis, supply chain management**.
    - Industries: **Banking, Manufacturing, Retail, Healthcare, Government**.
- **Big Data:**
    - **Fraud detection, personalized marketing, real-time customer insights, IoT analytics**.
    - Industries: **E-commerce, Social Media, AI-driven Healthcare, Smart Cities, Autonomous Vehicles**.

12. Tools & Technologies Used

- **Traditional BI:**
    - SQL-based databases: **Oracle, MySQL, PostgreSQL, Microsoft SQL Server**.
    - BI tools: **Tableau, Power BI, SAP BusinessObjects, IBM Cognos, QlikView**.
- **Big Data:**
    - Distributed computing: **Apache Hadoop, Apache Spark, AWS EMR**.
    - NoSQL databases: **MongoDB, Cassandra, Google BigQuery, Amazon DynamoDB**.

Data Warehouse Environments

A Data Warehouse (DW) is a structured system that consolidates and organizes business data for analysis and decision-making.

Key Features of Data Warehouses:

- **Domain-Centric: Data is structured around specific business areas such as sales, finance, or supply chain.**
- **Data Integration: Combines information from multiple sources into a unified structure.**
- **Time-Stamped Records: Stores historical data for trend analysis.**
- **Immutable Storage: Once stored, data remains unchanged and is updated through batch processes.**

Components of a Data Warehouse:

- **Source Systems: Includes databases, enterprise applications, and external data feeds.**
- **ETL Process (Extract, Transform, Load):**
    - **Extracts data from various systems.**
    - **Transforms it into a consistent format.**
    - **Loads it into the central warehouse.**
- **Storage Layer: Structured databases (e.g., Teradata, Snowflake, Amazon Redshift).**

- **Metadata Management: Maintains records about data origin, transformations, and schema.**
- **Query & Reporting Tools: Platforms like Power BI, Tableau, and SAP BusinessObjects facilitate business analysis.**

Advantages of Data Warehousing:

Well-structured data ensures reliability for decision-making.
Optimized for analytical queries, making insights easier to derive.
Helps businesses track performance over time through historical records.
Provides a single source of truth for organizations.

Challenges of Traditional Data Warehousing:

Scaling up requires expensive infrastructure.
Rigid schema structure limits flexibility with new data types.
Batch processing leads to delays in data availability.
Inability to handle vast amounts of unstructured data.

A Typical Hadoop Environment

Hadoop is an open-source ecosystem designed to process vast datasets by distributing tasks across multiple machines.

Core Elements of Hadoop:

- **HDFS (Hadoop Distributed File System):**
  - **Splits large datasets into blocks stored across multiple nodes.**
  - **Ensures redundancy to prevent data loss.**
- **MapReduce:**
  - **A processing framework that divides tasks into smaller chunks and executes them in parallel.**
- **YARN (Yet Another Resource Negotiator):**
  - **Manages job scheduling and system resources.**
- **Hadoop Ecosystem Tools:**
  - **Hive: SQL-like querying for structured data.**

- **Pig: Used for data transformation with a high-level scripting language.**
- **HBase: NoSQL database for real-time data access.**
- **Spark: A fast, in-memory processing engine.**
- **Kafka & Flink: Used for real-time data streaming and analytics.**

Benefits of Hadoop Environments:

Designed for scalability and high-volume data processing.
More cost-effective than traditional enterprise data solutions.
Suitable for diverse data types, including structured, semi-structured, and unstructured information.
Built-in redundancy prevents single points of failure.

Limitations of Hadoop:

Complex deployment and maintenance.
High-latency batch processing is unsuitable for real-time analytics.
Requires specialized skills to manage and optimize.

How Big Data is Transforming Data Storage & Analytics

The rise of Big Data technologies is reshaping how businesses store, process, and analyze information.

Shifts in Data Handling Due to Big Data:

- **Distributed vs. Centralized Storage:**
  - **Traditional data warehouses rely on centralized relational databases.**
  - **Big Data systems leverage distributed file storage and cloud-based architectures.**
- **Real-Time Processing Over Batch Systems:**
  - **Conventional BI tools operate on batch-processed data.**
  - **Big Data enables streaming analytics with real-time event processing.**
- **Handling Structured & Unstructured Data:**
  - **Data Warehouses are suited for structured SQL-based records.**

    - **Big Data ecosystems manage text, video, logs, sensor data, and multimedia.**

- **NoSQL & Hybrid Data Models:**
    - **Traditional BI tools work with SQL databases.**
    - **Big Data frameworks incorporate NoSQL (MongoDB, Cassandra) and hybrid architectures.**
- **Cloud & Cost-Efficient Scaling:**
    - **Warehouses require expensive infrastructure upgrades.**
    - **Big Data solutions scale dynamically with cloud computing (AWS, Azure, Google Cloud).**
- **AI & Machine Learning Integration:**
    - **Data Warehouses focus on reporting and descriptive analytics.**
    - **Big Data facilitates predictive analytics and automation through AI/ML models.**

Future Trends in Big Data Evolution:

Data Lakehouses: A fusion of Data Warehouses and Data Lakes to handle diverse data workloads.
Cloud-Based Analytics: Growing adoption of serverless and managed Big Data platforms.
Edge Computing: Bringing real-time data processing closer to devices.
AI-Powered Analytics: Automating business decisions through AI-driven insights.
Self-Service Data Platforms: Enabling business users to query and analyze data independently.

1. What is Big Data Analytics?

Big Data Analytics refers to **the process of examining large, complex datasets** to uncover hidden patterns, correlations, trends, and insights that aid decision-making. Unlike traditional data analysis, Big Data Analytics **handles vast amounts of structured, semi-structured, and unstructured data at scale**, often leveraging distributed computing, artificial intelligence, and machine learning.

Types of Big Data Analytics:

1. **Descriptive Analytics** – Summarizes past trends and events (e.g., dashboards, reports).
2. **Diagnostic Analytics** – Identifies reasons behind trends and anomalies.
3. **Predictive Analytics** – Uses historical data and machine learning to forecast future trends.
4. **Prescriptive Analytics** – Suggests optimal actions based on predictive models.

Real-World Applications:

Retail & E-commerce: Personalized recommendations, demand forecasting
Finance: Fraud detection, risk modeling, algorithmic trading
Healthcare: Medical imaging, predictive diagnosis, genomic analysis
Manufacturing: Predictive maintenance, supply chain optimization
Smart Cities: Traffic optimization, energy consumption monitoring

2. What Big Data Analytics is NOT

Despite its transformative impact, Big Data Analytics is often misunderstood. Here's what it is *not*:

Just About Large Data Volumes – While volume is a factor, Big Data Analytics is more about deriving **meaningful insights** from complex datasets.
Traditional Business Intelligence (BI) – BI provides historical insights, whereas Big Data Analytics incorporates **real-time analysis, AI, and predictive modeling**.
A One-Size-Fits-All Approach – There are multiple tools and methods tailored to different industries and needs.
Purely a Technology Upgrade – It involves a **strategic shift** in how data is collected, processed, and used for decision-making.
Only for Large Enterprises – Small and medium businesses also leverage Big Data solutions via **cloud-based platforms**.

Always Accurate – Data quality, bias, and ethical concerns must be addressed to ensure **meaningful and unbiased analytics**.

3. Why the Sudden Hype Around Big Data Analytics?

The growing buzz around Big Data Analytics is driven by several **technological advancements, market demands, and business transformations**.

Key Drivers of the Big Data Analytics Revolution:

1. Explosion of Data Generation

- Rapid increase in data from social media, IoT devices, transactions, sensors, and digital interactions.
- By 2025, the global data volume is projected to exceed **175 zettabytes**.

2. Advancements in AI & Machine Learning

- AI-driven analytics automates pattern recognition, anomaly detection, and decision-making.
- NLP (Natural Language Processing) enables sentiment analysis and chatbot interactions.

3. Cloud Computing & Storage Innovations

- Cloud platforms (AWS, Google Cloud, Azure) provide **on-demand, scalable data storage**.
- Data Lakehouses combine **structured data warehousing with unstructured data lakes**.

4. Demand for Real-Time Decision-Making

- Businesses need **instant insights** for customer engagement, fraud detection, and logistics.
- Streaming analytics platforms like **Apache Kafka and Apache Flink** process data in milliseconds.

5. Cost Reduction in Data Processing

- Open-source frameworks (Hadoop, Spark) lower barriers to entry.
- Cloud-native solutions reduce infrastructure costs.

6. Competitive Advantage for Businesses

- Companies leveraging Big Data Analytics outperform competitors by making **data-driven decisions**.

- Personalized marketing, dynamic pricing, and predictive maintenance boost profitability.

7. Regulatory & Compliance Needs

- Industries require **advanced analytics** for compliance with GDPR, HIPAA, and financial regulations.
- Risk management and anomaly detection prevent **fraud, cyber threats, and financial crimes**.

Types of Big Data Analytics

Big Data Analytics is categorized into four major types based on the nature of insights it provides:

Descriptive Analytics
Diagnostic Analytics
Predictive Analytics
Prescriptive Analytics

Each type serves a distinct purpose in analyzing data, from summarizing historical patterns to making real-time, AI-driven decisions.

Descriptive Analytics (What happened?)

Definition:

- Descriptive Analytics focuses on summarizing past data and presenting it in an understandable format.
- It provides insights into trends, patterns, and anomalies through dashboards, reports, and visualizations.
- Uses techniques like **data aggregation, clustering, and reporting**.

Key Features:
Summarizes historical data using reports, charts, and key performance indicators (KPIs).
Provides a static snapshot of business performance over a period.
Does not explain causes but highlights trends.

Examples:
E-commerce Sales Reports – Online platforms like Amazon analyze past sales, customer behavior, and website visits to identify peak shopping seasons.
Hospital Readmission Rates – Healthcare organizations analyze patient records to report trends in hospital readmissions, helping to assess the quality of care.

Common Tools Used:
Tableau, Power BI, Google Analytics, SQL-based reporting tools

Diagnostic Analytics (Why did it happen?)

Definition:

- Diagnostic Analytics digs deeper into past data to determine **causes and correlations** behind trends and anomalies.
- It uses techniques like **drill-down analysis, data mining, correlation analysis, and machine learning**.

Key Features:
Identifies root causes of trends and anomalies.
Uses statistical techniques to find patterns and relationships.
Answers "why" questions about business performance.

Examples:
Customer Churn Analysis – Telecom companies (e.g., Verizon, AT&T) analyze why customers switch to competitors by examining call drop rates, service complaints, and billing issues.
Healthcare Diagnostics – Hospitals analyze electronic medical records to determine why certain treatments lead to better patient outcomes.

Common Tools Used:
SAS, Python (Pandas, NumPy), R (ggplot2, dplyr), Apache Spark

Predictive Analytics (What is likely to happen?)

Definition:

- Predictive Analytics **forecasts future trends** using historical data, statistical modeling, and machine learning techniques.
- It is widely used for **risk assessment, demand forecasting, fraud detection, and personalized recommendations**.

Key Features:
Uses AI/ML algorithms (e.g., regression, time-series analysis, decision trees).
Forecasts future trends based on historical data patterns.
Enables proactive decision-making before an event occurs.

Examples:
Credit Score Prediction – Banks and financial institutions (e.g., FICO, Experian) use predictive analytics to assess customer creditworthiness and predict loan default risks.

Weather Forecasting – Meteorological agencies analyze historical climate data to predict upcoming storms, hurricanes, and temperature fluctuations.

Common Tools Used:
Python (Scikit-learn, TensorFlow), IBM SPSS, Apache Mahout, Google Cloud AI

Prescriptive Analytics (What should we do next?)

Definition:

- Prescriptive Analytics provides actionable recommendations based on predictive insights.
- It **goes beyond forecasting** by suggesting the best possible actions using optimization algorithms and AI-driven decision-making.

Key Features:
Suggests optimal decisions using simulation and optimization.
Uses reinforcement learning and advanced AI models for decision automation.
Common in autonomous systems like robotics, self-driving cars, and automated trading.

Examples:
Supply Chain Optimization – Companies like Walmart use prescriptive analytics to **automate inventory management** by analyzing demand fluctuations and logistics costs.
Autonomous Vehicles – Tesla's AI-powered system makes real-time decisions based on traffic data, pedestrian movements, and road conditions to ensure safe driving.

Common Tools Used:
Google OR-Tools, IBM Watson, Deep Reinforcement Learning frameworks

Comparison of the Four Types of Big Data Analytics

Analytics Type	Purpose	Techniques Used	Example
Descriptive	Summarizes past trends	Dashboards, reporting, visualization	Sales reports, website traffic analysis
Diagnostic	Explains reasons behind trends	Data mining, correlation analysis	Customer churn analysis, medical diagnostics
Predictive	Forecasts future trends	AI/ML models, statistical forecasting	Fraud detection, stock price prediction
Prescriptive	Recommends optimal actions	Decision optimization, reinforcement learning	Supply chain automation, autonomous vehicles

UNIT II

What is Hadoop?

Definition:

- **Apache Hadoop** is an **open-source framework** for storing and processing large datasets across **distributed computing clusters**.
- It follows the **MapReduce programming model** for parallel data processing.
- Developed by **Doug Cutting** and inspired by **Google's GFS and MapReduce papers**.
- Written in **Java** and maintained by the **Apache Software Foundation**.

Key Features:
Scalability – Can scale from a single server to thousands of machines.
Fault tolerance – Automatically recovers from node failures.
Cost-effectiveness – Uses commodity hardware to store and process data.
Flexibility – Supports structured, semi-structured, and unstructured data.

Main Use Cases:
Data Warehousing & ETL (Extract, Transform, Load)
Log Processing (e.g., analyzing website logs)
Machine Learning & AI (e.g., training models on large datasets)
Fraud Detection & Risk Management (e.g., financial services)

Hadoop Components (Core Modules)

Hadoop consists of **four primary components**:

1. Hadoop Distributed File System (HDFS)

Definition:

- A **fault-tolerant distributed storage system** designed to handle large files efficiently.
- Uses a **Master-Slave architecture** with **NameNode (Master)** and **DataNodes (Slaves)**.

How It Works:
Splits large files into smaller blocks (default: 128MB or 256MB).
Stores multiple copies (replication factor, default: 3).
Manages files efficiently across nodes.

Example:
A social media platform (e.g., **Facebook**) uses HDFS to store billions of images and videos across distributed clusters.

2. Yet Another Resource Negotiator (YARN)

Definition:

- A **resource management layer** in Hadoop that schedules tasks and allocates system resources.
- Separates resource management from data processing, improving efficiency.

Key Components:
ResourceManager – Manages system resources globally.
NodeManager – Manages resources on individual nodes.
ApplicationMaster – Oversees execution of applications.

Example:
Netflix uses YARN to allocate resources dynamically when processing user recommendations.

3. MapReduce (Processing Engine)

Definition:

- A **parallel processing model** that processes large datasets efficiently.
- Uses a **two-step approach**:
    1. **Map Phase** – Splits input data into smaller chunks for parallel processing.
    2. **Reduce Phase** – Aggregates and combines intermediate outputs into final results.

Example:
Twitter processes real-time tweets using **MapReduce** to analyze trending topics.

4. Hadoop Common (Utilities & Libraries)

Definition:

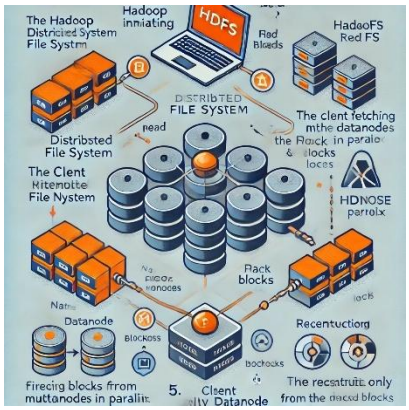
- A set of **shared utilities** and libraries for Hadoop's core modules.
- Ensures **compatibility** across various Hadoop versions.

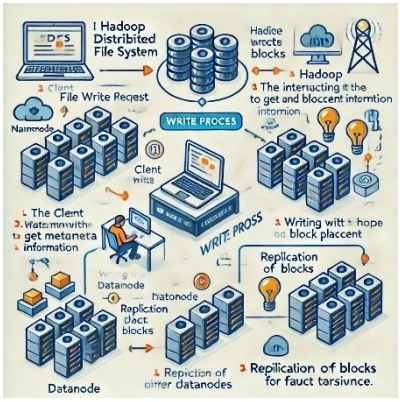
Key Features:
Provides **authentication** (Kerberos).

Supports **compression** for optimized storage.
Includes Java libraries for Hadoop applications.

Example:
A **financial firm** uses Hadoop Common libraries to integrate external data sources into their Hadoop cluster.

Hadoop Read and Write





HDFS Read Process (How Data is Retrieved from Hadoop)

Client Requests a File

- **The client contacts the NameNode to locate file blocks.**

NameNode Provides Block Locations

- **The NameNode does not store data but returns the list of DataNodes where each block resides.**

Client Reads Data from DataNodes

- **The client reads directly from DataNodes in parallel for efficiency.**
- **The client chooses the nearest DataNode for lower latency.**

Data is Reassembled

- **Once all blocks are fetched, the client reconstructs the complete file.**

Key Features:
Reads are optimized by fetching from the nearest DataNode.
NameNode only provides metadata; actual data is read from DataNodes.
Hadoop ensures fault tolerance – if a DataNode fails, the client reads from another replica.

HDFS Write Process (How Data is Stored in Hadoop)

Client Initiates File Write

- **The client wants to store a file in HDFS.**
- **The file is split into blocks (default 128MB or 256MB in Hadoop 3).**

Client Contacts the NameNode

- **The NameNode does not store data but maintains metadata about file locations.**
- **It provides block placement details for the file to be stored across DataNodes.**

DataNode Block Writing (Pipeline Write Process)

- **The client writes each block to a DataNode.**
- **The DataNode replicates the block to two other nodes (default replication factor: 3).**
- **The replication follows a pipeline fashion (DataNode 1 → DataNode 2 → DataNode 3).**

Acknowledgment & Completion

- **Once all DataNodes confirm replication, the NameNode updates metadata.**
- **The client receives a successful write confirmation.**

Key Features:
Data is stored in a fault-tolerant manner with replication.
NameNode only manages metadata, not actual data.
Pipeline replication ensures efficient storage across multiple nodes.

How is Hadoop Different from Traditional File Systems?

Feature	Traditional File Systems	HDFS
Storage Type	Centralized	Distributed
Fault Tolerance	Low	High (Replication)
Scalability	Limited	Linear Scalability
Processing	Sequential	Parallel
Metadata Storage	Within Filesystem	Stored in NameNode

Hadoop Ecosystem Components

Hadoop's ecosystem extends its capabilities with various tools:

Data Storage:

- **HBase** – NoSQL database for real-time read/write access.
- **Hive** – SQL-based query engine for structured data.

Data Processing:

- **Spark** – In-memory data processing engine (faster than MapReduce).
- **Pig** – High-level scripting language for analyzing data.

Data Ingestion:

- **Sqoop** – Transfers structured data from RDBMS to Hadoop.
- **Flume** – Collects and transfers log data to HDFS.

Workflow Management:

- **Oozie** – Manages and schedules Hadoop jobs.

Security & Governance:

- **Ranger** – Provides security and role-based access control.

Example:
Uber uses the Hadoop ecosystem (HDFS, Spark, Hive) to analyze ride data and optimize pricing models.

Advantages of Hadoop & HDFS over GFS and Other Distributed File Systems

Advantages of Hadoop Over Other Data Systems

Open-Source & Cost-Effective – Unlike Google File System (GFS), Hadoop is **open-source** and does not require proprietary hardware.

Scalability – Can **scale horizontally** by adding more nodes instead of upgrading expensive servers.
Supports Diverse Workloads – Unlike GFS, which is optimized for Google's internal services, Hadoop supports **machine learning, analytics, ETL, and real-time processing**.
Parallel Processing – Uses **MapReduce and Spark** to process data faster than traditional databases.

Advantages of HDFS Over GFS

1. Replication & Fault Tolerance:

- **HDFS replicates data across multiple nodes** (default: 3 copies), ensuring high availability.
- GFS also replicates, but **HDFS offers better handling of node failures**.

2. Data Locality Principle:

- In HDFS, computation moves **closer to data**, reducing network congestion.
- GFS does not prioritize local data processing as efficiently.

3. Cost-Effectiveness:

- HDFS runs on **commodity hardware**, while GFS is built for Google's infrastructure.
- This makes Hadoop **affordable for enterprises of all sizes**.

4. Block Size Optimization:

- **HDFS supports larger block sizes** (128MB+) for efficient storage and retrieval.
- GFS uses **smaller block sizes**, leading to higher metadata overhead.

5. Industry Adoption:

- Hadoop's **ecosystem is widely adopted** by industries (e.g., banking, retail, healthcare).
- GFS is proprietary and used only within Google's infrastructure.

Example:
Bank of America uses HDFS for real-time fraud detection instead of relying on traditional relational databases.

Core Hadoop Daemons (Hadoop 2 & 3)

Hadoop has **five primary daemons** that ensure smooth operations:

Daemon	Functionality	Runs on
NameNode	Manages metadata & file system namespace	Master Node
DataNode	Stores actual data blocks	Worker Nodes
ResourceManager	Allocates resources across the cluster	Master Node
NodeManager	Monitors and reports resource usage	Worker Nodes
JobHistoryServer	Stores historical job execution details	Master Node

---

Storage Layer Daemons (HDFS)

1. NameNode (Master Process for HDFS)

Functionality:

- Maintains **metadata** (file locations, permissions, directory structure) for HDFS.
- Acts as a **lookup service**, directing clients to appropriate **DataNodes**.
- Does **not store actual data**, only metadata.
- Uses a **checkpoint mechanism** for recovery.

Changes in Hadoop 3:
Erasure Coding replaces replication for improved storage efficiency.
Multiple NameNodes are now supported in HA (High Availability) mode.

Example:
If a user wants to read **/user/data.csv**, the NameNode tells them **which DataNodes store the file chunks**.

2. DataNode (Worker Process for HDFS)

Functionality:

- Stores **actual data blocks** and manages read/write operations.
- Periodically sends a **heartbeat** to the **NameNode** for health monitoring.
- Performs **block reporting** to inform the NameNode about available storage.

Changes in Hadoop 3:
Storage efficiency improved with **Erasure Coding**, reducing redundancy.
DataNodes can store data on heterogeneous storage types (SSD/HDD hybrid).

Example:
When a user uploads a file, the **NameNode breaks it into blocks**, and **DataNodes store these blocks**.

Resource Management Daemons (YARN)

3. ResourceManager (Master Process for YARN)

Functionality:

- Manages **cluster resources** (CPU, memory) for running applications.
- Works with the **Scheduler** to distribute tasks efficiently.
- Assigns tasks to **ApplicationMasters** to execute processing jobs.

Changes in Hadoop 3:
Federation Support – Multiple **ResourceManagers** can operate in parallel for scalability.
GPU & FPGA support – Improved hardware acceleration for ML tasks.

Example:
If multiple users submit jobs, the **ResourceManager allocates memory & CPU** to ensure fair resource distribution.

4. NodeManager (Worker Process for YARN)

Functionality:

- Runs on every worker node, **monitoring CPU and memory usage**.
- Communicates with the **ResourceManager** for task execution.
- Manages **container lifecycles** (executing MapReduce or Spark tasks).

Changes in Hadoop 3:
Supports GPU scheduling, allowing deep learning workloads.
Improved container reuse for better efficiency.

Example:
If a **Spark job** requires 2GB RAM, the **NodeManager ensures that enough resources are available** before execution.

Job Execution Daemon

5. JobHistoryServer

Functionality:

- Stores **historical data** about completed jobs.
- Allows users to **analyze past job execution details** (e.g., runtime, failures, logs).

Changes in Hadoop 3:
Optimized log storage reduces overhead on large clusters.

Example:
A **data engineer** can check the **JobHistoryServer** to debug a failed MapReduce job.

Additional Hadoop 3 Enhancements

1. Erasure Coding in HDFS:
Reduces storage overhead by **50% compared to replication**.
Example: Instead of storing **3 copies (replication factor 3)**, Erasure Coding stores **redundant parity bits**.

2. Containerization & GPU Support in YARN:
Supports Docker containers, making application management more flexible.
GPUs are **now allocated dynamically** for AI/ML workloads.

3. Multiple Active NameNodes:
Hadoop 3 supports **more than two NameNodes**, improving **fault tolerance**.

4. HDFS Heterogeneous Storage Support:
Allows **tiered storage**, where **hot data is stored on SSDs** and **cold data on HDDs**.

Hadoop can be installed in different modes depending on the use case and environment. Below are the **three main modes of Hadoop installation** along with their differences:

Standalone Mode (Local Mode)

- **Definition:**

  - The simplest mode where Hadoop runs on a **single machine** without using HDFS.
  - Mostly used for **testing and debugging**.

- **Key Characteristics:**

  - No **distributed storage** (HDFS is not used).
  - All components (NameNode, DataNode, ResourceManager, NodeManager) run as **single Java processes**.
  - Input and output are handled using **local file system** instead of HDFS.

- **Advantages:**

Quick setup, minimal configuration.
Ideal for testing **MapReduce** programs before deploying them in clusters.

- **Disadvantages:**

No distributed processing or storage.
Cannot handle large-scale Big Data workloads.

- **Usage:**

  - Used by developers for **testing and debugging** Hadoop applications before running them on a cluster.

Pseudo-Distributed Mode (Single-Node Cluster)

- **Definition:**

  - Hadoop runs on **a single machine**, but all daemons (NameNode, DataNode, ResourceManager, NodeManager, etc.) run as separate processes.
  - Mimics a real Hadoop cluster but with **only one node**.

- **Key Characteristics:**

  - Uses **HDFS for storage** instead of the local file system.
  - Supports distributed computing (though limited to one node).

- Requires configuration of **core-site.xml, hdfs-site.xml, mapred-site.xml and yarn-site.xml**.

🔹 **Advantages:**
Simulates a real Hadoop cluster.
Allows developers to test **HDFS and YARN** before deploying on a full cluster.

🔹 **Disadvantages:**
Not suitable for real-world large-scale applications.
Performance is limited to a single machine.

🔹 **Usage:**

- Used for **small-scale testing and learning** Hadoop before moving to production.

Fully Distributed Mode (Multi-Node Cluster)

🔹 **Definition:**

- The production mode where Hadoop runs on **multiple machines (nodes)**.
- Nodes are categorized into **Master Nodes (NameNode, ResourceManager) and Worker Nodes (DataNodes, NodeManagers)**.

🔹 **Key Characteristics:**

- Supports **HDFS for distributed storage**.
- Uses **YARN (Yet Another Resource Negotiator)** for resource management.
- Handles **large-scale parallel processing** efficiently.
- Ensures **fault tolerance and high availability** with replication.

🔹 **Advantages:**
Fully distributed **Big Data processing**.
Scales horizontally by adding more nodes.
Fault-tolerant and optimized for **real-world workloads**.

🔹 **Disadvantages:**
Requires **complex setup and configuration**.
Needs **high hardware resources** and infrastructure management.

🔹 **Usage:**

- Used in **real-world production** environments for **Big Data processing**.

- Implemented by companies dealing with **huge datasets (e.g., Facebook, Amazon, Google, Netflix, etc.)**.

🔹 Comparison of Hadoop Modes

Feature	**Standalone Mode**	**Pseudo-Distributed Mode**	**Fully Distributed Mode**
Number of Nodes	1	1	Multiple
HDFS Usage	No	Yes	Yes
Daemons Running	Single Java Process	Separate Processes	Distributed Across Nodes
Resource Management	Local FS	YARN (Simulated)	YARN
Use Case	Debugging & Testing	Small-scale Testing	Large-scale Production
Fault Tolerance	No	No	Yes
Performance	Low	Medium	High

Hadoop Installation Guide (Linux & Windows)

---

1. Installing Hadoop in Standalone Mode

- **On Linux (Ubuntu/Debian/CentOS)**

Step 1: Install Java (JDK 8 or higher)

```
sudo apt update
sudo apt install openjdk-8-jdk -y
java -version
```

Verification: Ensure Java is installed by running java -version. Expected output:

```
openjdk version "1.8.0_xxx"
OpenJDK Runtime Environment...
```

Step 2: Download Hadoop

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.1/hadoop-3.3.1.tar.gz
tar -xvzf hadoop-3.3.1.tar.gz
mv hadoop-3.3.1 ~/hadoop
```

Step 3: Configure Environment Variables

Add the following lines to ~/.bashrc:

```
export HADOOP_HOME=~/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export JAVA_HOME=$(readlink -f /usr/bin/java | sed "s:bin/java::")
```

Then run:

```
source ~/.bashrc
```

Step 4: Run Standalone Hadoop

```
hadoop version
```

hadoop fs -ls /

Verification: Run hadoop version and check Hadoop version details.

---

2. Installing Hadoop in Pseudo-Distributed Mode

- **On Linux**

Step 1: Configure Hadoop Core Files

Modify /home/user/hadoop/etc/hadoop/core-site.xml:

```
<configuration>
 <property>
  <name>fs.defaultFS</name>
  <value>hdfs://localhost:9000</value>
 </property>
</configuration>
```

- **Explanation:**
  - fs.defaultFS → Defines the **default file system** (HDFS in this case) and its address.

Modify /home/user/hadoop/etc/hadoop/hdfs-site.xml:

```
<configuration>
 <property>
  <name>dfs.replication</name>
  <value>1</value>
 </property>
 <property>
  <name>dfs.namenode.name.dir</name>
  <value>file:/home/user/hadoop_data/hdfs/namenode</value>
 </property>
 <property>
  <name>dfs.datanode.data.dir</name>
  <value>file:/home/user/hadoop_data/hdfs/datanode</value>
 </property>
</configuration>
```

- **Explanation:**

- dfs.replication → Defines **number of copies** of data blocks.
- dfs.namenode.name.dir → Directory to **store NameNode metadata**.
- dfs.datanode.data.dir → Directory for **DataNode storage**.

Modify /home/user/hadoop/etc/hadoop/mapred-site.xml:

```
<configuration>
 <property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
 </property>
</configuration>
```

🔹 **Explanation:**

- mapreduce.framework.name → Defines MapReduce execution engine (YARN).

Modify /home/user/hadoop/etc/hadoop/yarn-site.xml:

```
<configuration>
 <property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
 </property>
</configuration>
```

🔹 **Explanation:**

- yarn.nodemanager.aux-services → Enables **MapReduce shuffle service**.

Modify /home/user/hadoop/etc/hadoop/masters file:

```
master
```

Modify /home/user/hadoop/etc/hadoop/workers file:

```
worker1
worker2
```

Step 2: Setup Passwordless SSH

```
ssh-keygen -t rsa -P ""
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost
```

Verification: Should be able to SSH into localhost without a password. 📸

Step 3: Format NameNode

```
hdfs namenode -format
```

Step 4: Start Hadoop Services

```
start-dfs.sh
start-yarn.sh
mr-jobhistory-daemon.sh start historyserver
jps  # Check if NameNode, DataNode, ResourceManager, JobHistoryServer are running
```

Verification: Run jps and check if necessary daemons are running. 📸

Troubleshooting Guide

Java Not Found Error

Solution: Ensure Java is installed and JAVA_HOME is set correctly:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

SSH Connection Issues

Solution:

- Ensure sshd is running: sudo systemctl restart ssh
- Check permissions: chmod 600 ~/.ssh/authorized_keys

NameNode Format Issues

Solution: Remove old metadata and reformat:

```
rm -r /home/user/hadoop_data/hdfs/namenode/*
hdfs namenode -format
```

DataNode Failing to Start

Solution: Check logs in logs/ directory:

tail -f $HADOOP_HOME/logs/hadoop-*.log

Feature	RDBMS	Hadoop
Data Variety	Mainly for Structured data	Used for Structured, Semi-Structured, and Unstructured data
Data Storage	Average size data (GBS)	Use for large data sets (Tbs and Pbs)
Querying	SQL Language	HQL (Hive Query Language)
Schema	Required on write (static schema)	Required on reading (dynamic schema)
Speed	Reads are fast	Both reads and writes are fast
Cost	License	Free
Use Case	OLTP (Online transaction processing)	Analytics (Audio, video, logs, etc.), Data Discovery
Data Objects	Works on Relational Tables	Works on Key/Value Pair
Throughput	Low	High
Scalability	Vertical	Horizontal
Hardware Profile	High-End Servers	Commodity/Utility Hardware
Integrity	High (ACID)	Low

Key Challenges in Distributed Systems

Distributed systems offer scalability, fault tolerance, and performance benefits, but they also introduce significant complexities. Below are some of the most pressing challenges in distributed computing:

1. **Network Partitioning and Communication Failures**

One of the fundamental problems in distributed environments is network partitioning, where nodes lose connectivity with each other. This can lead to split-brain scenarios, where different parts of the system operate independently, causing inconsistencies. Ensuring network resilience and using strategies like failure detectors help mitigate such issues.

2. **Data Replication and Consistency Trade-offs**

Maintaining data consistency across multiple nodes is difficult, especially when trying to balance availability and fault tolerance. Systems must choose between eventual consistency (where updates propagate over time) and strong consistency (where all replicas remain synchronized). The CAP theorem dictates that in a partitioned network, a system can only guarantee two out of three properties: Consistency, Availability, and Partition Tolerance.

3. **Fault Tolerance and Self-Healing Mechanisms**

Distributed systems must continue functioning even when individual nodes fail. Achieving fault tolerance requires redundancy, failure detection, and automated recovery mechanisms. Techniques such as checkpointing, leader election, and self-healing architectures ensure that failures do not bring down the entire system.

4. **Synchronization, Concurrency, and Coordination**

With multiple nodes operating in parallel, coordinating access to shared resources becomes a challenge. Without proper synchronization mechanisms, race conditions and deadlocks can occur. Distributed systems use consensus protocols (e.g., Paxos, Raft) and distributed locking mechanisms (e.g., ZooKeeper) to maintain order and prevent data corruption.

5. **Scalability and Load Distribution**

As workloads grow, distributed systems must scale efficiently while balancing loads across available resources. Poor load balancing can lead to hotspots, where some nodes become overloaded while others remain underutilized. Dynamic load balancing techniques, auto-scaling, and horizontal scaling strategies help manage varying workloads efficiently.

Strategies to Address Distributed System Challenges

To overcome these challenges, various approaches and best practices have been developed:

1. Consensus and Replication Strategies

Using consensus algorithms such as Paxos, Raft, and Two-Phase Commit (2PC) ensures that distributed nodes agree on data updates while maintaining fault tolerance. Replication strategies like leader-follower replication and quorum-based reads/writes enhance data reliability.

2. Quorum-Based Techniques for Consistency

In distributed databases, quorum-based techniques help maintain consistency even when network partitions occur. Systems like Amazon DynamoDB and Apache Cassandra use quorum reads and writes to achieve a balance between consistency and availability.

3. Circuit Breaker Pattern for Fault Isolation

The circuit breaker pattern prevents cascading failures by detecting failures in services and temporarily blocking interactions with unresponsive components. This allows services to recover gracefully instead of causing system-wide outages.

4. Asynchronous Communication and Event-Driven Architectures

Reducing direct dependencies between services through asynchronous messaging (e.g., Kafka, RabbitMQ, and event-driven microservices) enhances system resilience and prevents bottlenecks.

5. Observability: Distributed Tracing and Monitoring

To diagnose failures effectively, distributed systems require comprehensive monitoring and tracing. Tools like Jaeger, Prometheus, and OpenTelemetry provide insights into request flows, bottlenecks, and latency issues across microservices.

Create a customfileinputformat using a customfilerecordreader and process wordcount.

CustomFileInputFormat.java

```
import java.io.IOException;


import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.InputSplit;

import org.apache.hadoop.mapreduce.RecordReader;

import org.apache.hadoop.mapreduce.TaskAttemptContext;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;


public class CustomFileInputFormat extends FileInputFormat<LongWritable,Text>{


	@Override

	public RecordReader<LongWritable, Text> createRecordReader(

			InputSplit split, TaskAttemptContext context) throws IOException,

			InterruptedException {


		return new CustomLineRecordReader();

	}

}
```

CustomLineRecordReader.java

```
import java.io.IOException;


import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FSDataInputStream;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.InputSplit;
import org.apache.hadoop.mapreduce.RecordReader;
import org.apache.hadoop.mapreduce.TaskAttemptContext;
import org.apache.hadoop.mapreduce.lib.input.FileSplit;
import org.apache.hadoop.util.LineReader;


public class CustomLineRecordReader extends RecordReader<LongWritable, Text> {


private long start;
private long pos;
private long end;
```

```
private LineReader in;

private int maxLineLength;

private LongWritable key = new LongWritable();

private Text value = new Text();


private static final Log LOG = LogFactory.getLog(CustomLineRecordReader.class);


/**
 * This method takes as arguments the map task's assigned InputSplit and
 * TaskAttemptContext, and prepares the record reader. For file-based input
 * formats, this is a good place to seek to the byte position in the file to
 * begin reading.
 */
@Override
public void initialize(InputSplit genericSplit, TaskAttemptContext context) throws IOException {


        // This InputSplit is a FileInputSplit
        FileSplit split = (FileSplit) genericSplit;


        // Retrieve configuration, and Max allowed
        // bytes for a single record
        Configuration job = context.getConfiguration();
```

```
        this.maxLineLength = job.getInt("mapred.linerecordreader.maxlength",
Integer.MAX_VALUE);



        // Split "S" is responsible for all records

        // starting from "start" and "end" positions

        start = split.getStart();

        end = start + split.getLength();



        // Retrieve file containing Split "S"

        final Path file = split.getPath();

        FileSystem fs = file.getFileSystem(job);

        FSDataInputStream fileIn = fs.open(split.getPath());



        // If Split "S" starts at byte 0, first line will be processed

        // If Split "S" does not start at byte 0, first line has been already

        // processed by "S-1" and therefore needs to be silently ignored

        boolean skipFirstLine = false;

        if (start != 0) {

                skipFirstLine = true;

                // Set the file pointer at "start - 1" position.

                // This is to make sure we won't miss any line

                // It could happen if "start" is located on a EOL

                --start;
```

```
                fileIn.seek(start);
        }

        in = new LineReader(fileIn, job);

        // If first line needs to be skipped, read first line
        // and stores its content to a dummy Text
        if (skipFirstLine) {
                Text dummy = new Text();
                // Reset "start" to "start + line offset"
                start += in.readLine(dummy, 0, (int) Math.min((long) Integer.MAX_VALUE, end -
start));
        }

        // Position is the actual start
        this.pos = start;

}

/**
 * Like the corresponding method of the InputFormat class, this reads a
 * single key/ value pair and returns true until the data is consumed.
 */
```

```
@Override
public boolean nextKeyValue() throws IOException {

        // Current offset is the key
        key.set(pos);

        int newSize = 0;

        // Make sure we get at least one record that starts in this Split
        while (pos < end) {

                // Read first line and store its content to "value"
                newSize = in.readLine(value, maxLineLength, Math.max((int)
Math.min(Integer.MAX_VALUE, end - pos), maxLineLength));

                // No byte read, seems that we reached end of Split
                // Break and return false (no key / value)
                if (newSize == 0) {
                        break;
                }

                // Line is read, new position is set
                pos += newSize;
```

```
            // Line is lower than Maximum record line size
            // break and return true (found key / value)
            if (newSize < maxLineLength) {
                        break;
            }

            // Line is too long
            // Try again with position = position + line offset,
            // i.e. ignore line and go to next one
            // TODO: Shouldn't it be LOG.error instead ??
            LOG.info("Skipped line of size " + newSize + " at pos " + (pos - newSize));
}



if (newSize == 0) {
            // We've reached end of Split
            key = null;
            value = null;
            return false;
} else {
            // Tell Hadoop a new line has been found
            // key / value will be retrieved by
```

```
                // getCurrentKey getCurrentValue methods
                return true;
        }
}

/**
 * This methods are used by the framework to give generated key/value pairs
 * to an implementation of Mapper. Be sure to reuse the objects returned by
 * these methods if at all possible!
 */
@Override
public LongWritable getCurrentKey() throws IOException,
                InterruptedException {
        return key;
}

/**
 * This methods are used by the framework to give generated key/value pairs
 * to an implementation of Mapper. Be sure to reuse the objects returned by
 * these methods if at all possible!
 */
@Override
public Text getCurrentValue() throws IOException, InterruptedException {
```

```
        return value;
}

/**
 * Like the corresponding method of the InputFormat class, this is an
 * optional method used by the framework for metrics gathering.
 */
@Override
public float getProgress() throws IOException, InterruptedException {
        if (start == end) {
                return 0.0f;
        } else {
                return Math.min(1.0f, (pos - start) / (float) (end - start));
        }
}

/**
 * This method is used by the framework for cleanup after there are no more
 * key/value pairs to process.
 */
@Override
public void close() throws IOException {
        if (in != null) {
```

```
                in.close();
        }
}

}
```

MapClass.java

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

/**
 * Map Class which extends MaReduce.Mapper class
 * Map is passed a single line at a time, it splits the line based on space
 * and generated the token which are output by map with value as one to be consumed
 * by reduce class
 * @author Raman
 */
public class MapClass extends Mapper<LongWritable, Text, Text, IntWritable>{
```

```
        private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    /**
     * map function of Mapper parent class takes a line of text at a time
     * splits to tokens and passes to the context as word along with value as one
     */
        @Override
        protected void map(LongWritable key, Text value,
                        Context context)
                        throws IOException, InterruptedException {

                String line = value.toString();
                StringTokenizer st = new StringTokenizer(line," ");

                while(st.hasMoreTokens()){
                        word.set(st.nextToken());
                        context.write(word,one);
                }

        }
}
```

ReduceClass.java

```
import java.io.IOException;
import java.util.Iterator;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

/**
 * Reduce class which is executed after the map class and takes
 * key(word) and corresponding values, sums all the values and write the
 * word along with the corresponding total occurances in the output
 *
 * @author Raman
 */
public class ReduceClass extends Reducer<Text, IntWritable, Text, IntWritable>{

	/**
	 * Method which performs the reduce operation and sums
	 * all the occurrences of the word before passing it to be stored in output
	 */
	@Override
	protected void reduce(Text key, Iterable<IntWritable> values,
```

```
                Context context)
                throws IOException, InterruptedException {

        int sum = 0;
        Iterator<IntWritable> valuesIt = values.iterator();

        while(valuesIt.hasNext()){
                sum = sum + valuesIt.next().get();
        }

        context.write(key, new IntWritable(sum));
    }
}
```

WordCount.java (Driver Code)

```
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
```

```
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

/**
 * The entry point for the WordCount example,
 * which setup the Hadoop job with Map and Reduce Class
 *
 * @author Raman
 */
public class WordCount extends Configured implements Tool{

	/**
	 * Main function which calls the run method and passes the args using ToolRunner
	 * @param args Two arguments input and output file paths
	 * @throws Exception
	 */
	public static void main(String[] args) throws Exception{
		int exitCode = ToolRunner.run(new WordCount(), args);
		System.exit(exitCode);
	}
```

```
	/**
	 * Run method which schedules the Hadoop Job
	 * @param args Arguments passed in main function
	 */
	public int run(String[] args) throws Exception {
		if (args.length != 2) {
			System.err.printf("Usage: %s needs two arguments <input> <output>
files\n",
					getClass().getSimpleName());
			return -1;
		}

		//Initialize the Hadoop job and set the jar as well as the name of the Job
		Job job = new Job();
		job.setJarByClass(WordCount.class);
		job.setJobName("WordCounter");

		//Add input and output file paths to job based on the arguments passed
		CustomFileInputFormat.addInputPath(job, new Path(args[0]));
		job.setInputFormatClass(CustomFileInputFormat.class);

		FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

```
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        //Set the MapClass and ReduceClass in the job
        job.setMapperClass(MapClass.class);
        job.setReducerClass(ReduceClass.class);

        //Wait for the job to complete and print if the job was successful or not
        int returnValue = job.waitForCompletion(true) ? 0:1;

        if(job.isSuccessful()) {
            System.out.println("Job was successful");
        } else if(!job.isSuccessful()) {
            System.out.println("Job was not successful");
        }

        return returnValue;
    }
}
```